Jyotsna Kasibhotla - 23102B0078

COMMUNITY HEALTH SURVEY SYSTEM

1. Introduction

1.1 Week 5 Objective

The objective of Week 5 was to implement the core survey management functionality of the Community Health Survey System. The following features were implemented:

- Create a new health survey
- Validate mandatory survey fields
- Automatically assign DRAFT status to newly created surveys
- View all surveys
- View an individual survey
- Update an existing survey
- Search surveys using ID and text fields
- Display a suitable message when no records are found
- Preserve the existing survey status during updates

The implementation was developed using Spring Boot, Spring Data JPA, Thymeleaf, MySQL, and Maven.

2. Week 5 Scope

Feature	Status
Create Survey	☑ Completed
Survey Validation	☑ Completed
View Surveys	☑ Completed
View Individual Survey	☑ Completed
Update Survey	☑ Completed
Search by Survey ID	☑ Completed
Search by Name	☑ Completed
Search by Location/Health Condition	☑ Completed
No Search Results Message	☑ Completed
Status Preservation During Update	☑ Completed

3. Technologies Used

Technology	Purpose

Java 17	Application development
Spring Boot 4.1.1	Backend framework
Spring MVC	Request handling
Spring Data JPA	Database operations
Hibernate	ORM
Thymeleaf	Frontend templates
MySQL	Relational database
Maven	Build and dependency management
Jakarta Validation	Input validation
VS Code	Development environment
Git/GitHub	Version control

4. Survey Data Model

The Survey entity represents a community health survey record stored in the MySQL database.

4.1 Survey Fields

Field	Description
id	Unique survey identifier
name	Name of the surveyed person
age	Age of the person
gender	Gender
location	Survey location
healthCondition	Reported health condition
surveyDate	Date on which survey was conducted
status	Current survey workflow status

4.2 Survey Status

The system supports the following statuses:

- DRAFT
- SUBMITTED
- VERIFIED
- CLOSED

For Week 5, newly created surveys are automatically assigned DRAFT. The complete workflow transitions will be implemented in Week 6.

5. Create Survey Implementation

5.1 Create Survey Flow

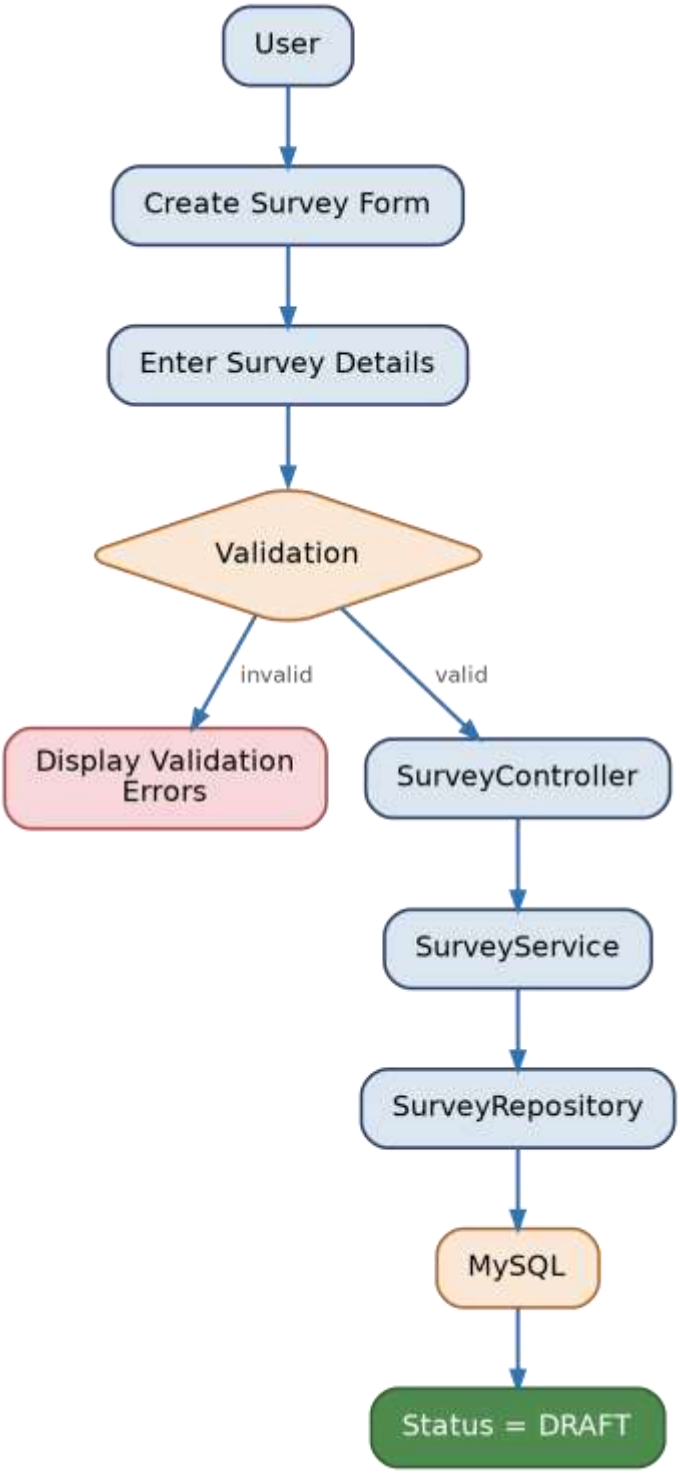

Figure 1: Create survey flow — from user input to DRAFT status

5.2 Implementation

The create functionality was implemented using:

- SurveyController

- SurveyService
- SurveyRepository
- Survey entity
- Thymeleaf create form

When the user submits valid survey information, the request is processed by the controller and passed to the service layer. The service automatically assigns the initial status DRAFT before saving the survey to MySQL.

6. Input Validation

Validation was added to ensure that invalid or incomplete survey records cannot be submitted.

6.1 Validation Rules

Field	Validation
Name	Required
Age	Required, between 1 and 120
Gender	Required
Location	Required
Health Condition	Required
Survey Date	Required

Examples of validation annotations used include:

- @NotBlank
- @NotNull
- @Min
- @Max

If validation fails, the user remains on the form and appropriate validation messages are displayed. Example:

- Name is required
- Age must be between 1 and 120
- Location is required

7. View Survey

The application supports viewing all available surveys.

```
http://localhost:8081/surveys
```

The survey listing displays the stored survey records. Users can select a survey to view its complete details.

7.1 Individual Survey

Example URL pattern:

```
/surveys/{id}
```

For example: /surveys/1

8. Update Survey

The update functionality allows existing survey information to be modified.

8.1 Update Flow

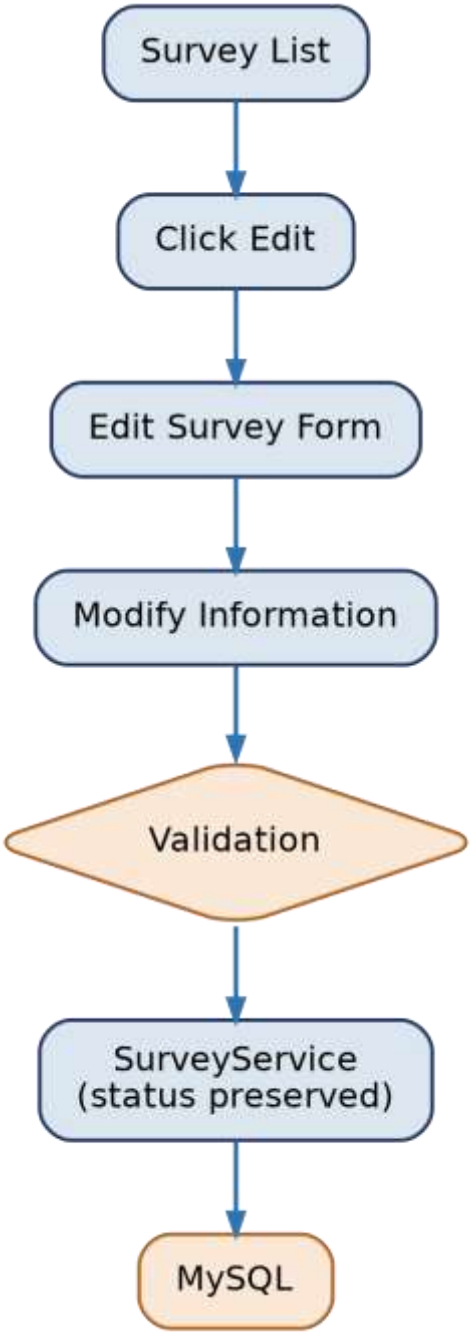

Figure 2: Update survey flow — status is preserved, not reset

8.2 Updatable Fields

- Name
- Age

- Gender
- Location
- Health condition
- Survey date

9. Search Survey

A search facility was implemented to allow users to find survey records. The search supports:

1. Survey ID

Example: search "5" returns survey ID 5.

2. Name

Example: search "Rahul". The search is case-insensitive — rahul, Rahul and RAHUL can all match the same record.

3. Location

Example: search "Bangalore".

4. Health Condition

Example: search "Diabetes".

10. Search Implementation

The repository uses a custom query to perform text-based searches. The query checks Name, Location and Health Condition using case-insensitive matching. Numeric search values are treated as survey IDs.

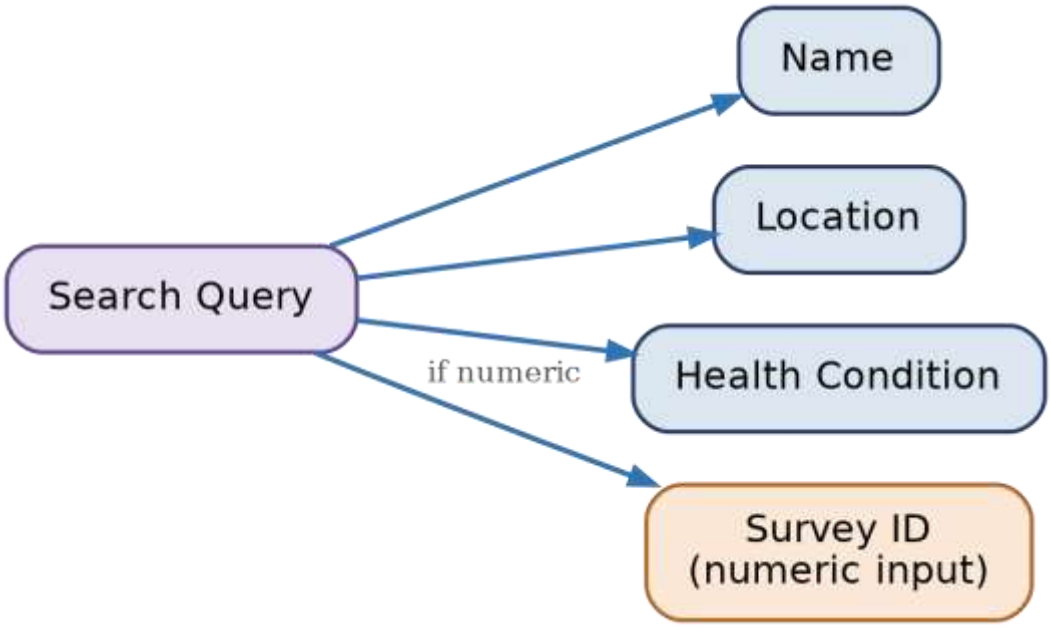

Figure 3: Search query — matched fields

11. Application Architecture

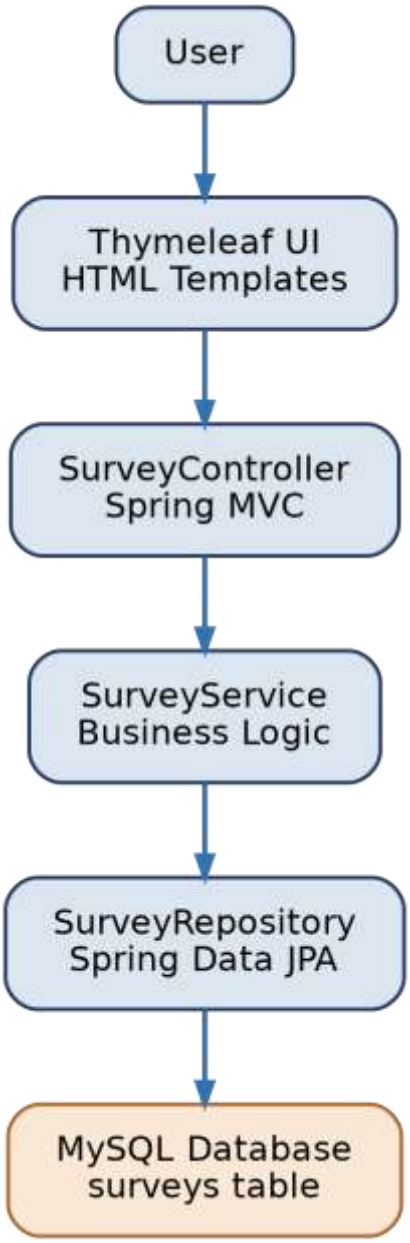

Figure 4: Application architecture — User to MySQL

12. Project Structure

Current relevant project structure:

```
src
└── main
    ├── java
    │   └── com.communityhealth.survey
    │       ├── controller
    │       │   └── SurveyController.java
    │       ├── entity
```

```
    │       │   ├── Survey.java
    │       │   └── User.java
    │       ├── enums
    │       │   └── SurveyStatus.java
    │       ├── repository
    │       │   └── SurveyRepository.java
    │       ├── service
    │       │   └── SurveyService.java
    │       └── CommunityHealthSurveyApplication.java
    │
    └── resources
        ├── static
        ├── templates
        │   ├── home.html
        │   └── surveys
        │       ├── create.html
        │       ├── edit.html
        │       ├── list.html
        │       └── view.html
        └── application.properties
```

13. Testing

Test	Expected Result	Result
Application startup	Application starts successfully	☑ PASS
Create valid survey	Survey saved as DRAFT	☑ PASS
Invalid survey submission	Validation errors displayed	☑ PASS
View surveys	Survey list displayed	☑ PASS
View individual survey	Survey details displayed	☑ PASS
Update survey	Updated data saved	☑ PASS
Search by ID	Matching survey returned	☑ PASS
Search by text	Matching surveys returned	☑ PASS
No search match	No-results message displayed	☑ PASS
Maven build	Build completes successfully	☑ PASS